1. Stacking

   • push(1): [1]

   • push(20): [1,20]

   • pop(): [1]

   • push(pop()*2): [2]

   • push(0): [2,0]

   • push(pop()/2): [2,0]

2. Queueing
   • push(2) [2]
   • push(pop()+4) [6]
   • push(5) [6,5]
   • push(pop()/2) [5,3]
   • pop() [5]
   • pop() []
3. Int findInDeque(int[] q, int y)

```
{
        int front = 0;
        int back = q.length – 1;
        while (front <= back)
        {
                If (q[front] == y)
                        return front;
                If (q[back] == y)
                        return back;
                front++;
                back--;
        {
        return -1;
}
```

The time complexity is O(n/2), as with each iteration in the while loop, the deque is checked from both ends inwards, making the maximum iterations n/2.

7. The balancedBracket's time complexity is O(n) where n is the number of brackets in the string; In the worst case, when the input is a balanced sequence, the loop will check every bracket once, and every iteration only executes constant operations. The space

complexity is also O(n). Aside from the input, there are three variables used: openStack, open, and c. In the worst case, if all brackets in the sequence are open brackets, every character in the input will be pushed to the OpenStack, so the stack would grow to size n. The other variables, c and open, use constant space.

The DecodeString solution's time and space complexity are both O(n + m), where n is the length of the input, and m is the length of the output. The input string is checked once, but when encountering a closing bracket, a substring will be added to the output by the preceding counting number. Additionally, in the worst case, the stacks store up to n elements, as well as the current string variable storing the decoded string of size m.

The PostfixToInfix solution's time and space complexity are both O(n), where n is the size of the input. In the worst case space-wise, that being all characters in the input are operators or parentheses, they would all be pushed onto the stack. Time-wise, all characters in the string would be evaluated exactly once.